Day 10

Object-Oriented Programming I

Classes & Objects

Phase	Day	Duration	Difficulty
Intermediate	10 of 30	2 hours	★★★■■

Session Goal

By the end of this session students will understand the core building blocks of Object-Oriented Programming in Python — defining classes, using the __**init**__ constructor, understanding **self**, distinguishing instance from class variables, and writing instance methods that read and modify object state.

Learning Outcomes

- Define a class using the **class** keyword with PascalCase naming
- Write an __**init**__ constructor that sets up initial object state
- Explain what **self** is and why it appears in every instance method
- Distinguish instance variables (**self.x**) from class variables
- Write multiple instance methods that interact via **self**
- Identify and fix the three most common OOP beginner mistakes
- Build a multi-method class from scratch (Library Book mini-challenge)

Prerequisites

Days 1–9 complete: variables, control flow, loops, functions,
data structures, strings, file I/O, and error handling.
Students should be comfortable writing multi-function scripts.

What Students Need

Python 3.12 environment with VS Code open
day10_oop_classes.py open in the editor
Terminal ready to run: python day10_oop_classes.py

SESSION TIMELINE

2-hour minute-by-minute guide

Time	Activity	Mode
0:00–0:10	Welcome & recap of Day 9 (error handling)	Discussion
0:10–0:25	Why OOP? Conceptual intro — blueprints vs objects	Lecture
0:25–0:40	Defining a class, __init__, self — live demo	Live coding
0:40–0:55	Instance variables vs class variables	Live coding
0:55–1:05	BREAK	—
1:05–1:20	Instance methods — Rectangle example	Live coding
1:20–1:40	Guided examples: MovieTicket, CricketPlayer, GroceryCart	Live coding
1:40–1:50	Common mistakes walkthrough	Discussion
1:50–2:00	Mini-challenge: Library Book System	Student work

Instructor Note

OOP is a mindset shift. Many students find it abstract at first.
Emphasise the analogy: class = blueprint, object = built thing.
Use Student and BankAccount — students relate to these immediately.
Encourage students to predict output BEFORE running each example.

STEP 1 — Why OOP? (0:10 – 0:25)

What to Say

"Until now, if we wanted to track a student's name, age and city, we'd use three separate variables — or a dictionary. That works, but what if we need 500 students, each with methods like greet() or calculate_gpa()? OOP lets us package data AND the functions that work on it into one tidy unit called a class."

Procedural vs OOP — side by side

```
# --- PROCEDURAL (what you know) ---
student_name = 'Priya'
student_age  = 20
def greet(name, age):
    print(f'Hi, I am {name}, {age} years old.')
greet(student_name, student_age)

# --- OOP (what you will learn today) ---
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age  = age
    def greet(self):
        print(f'Hi, I am {self.name}, {self.age} years old.')

priya = Student('Priya', 20)
priya.greet()          # Hi, I am Priya, 20 years old.
```

> **Key Point**
> A class is a BLUEPRINT. An object (instance) is what you BUILD from it.
> Think: Car class -> priya_car, arjun_car are two different objects.

STEP 2 — __init__ and self (0:25 – 0:40)

What to Say

"__init__ is Python's constructor — it runs automatically the moment you write Student('Priya', 20). Think of it as the factory setup line. And self? It's Python's way of saying 'this specific object right here'. When Priya calls greet(), Python passes Priya's object as the first argument automatically."

```
class Student:
    def __init__(self, name, age, city):
        self.name = name   # stored ON the object
        self.age  = age
        self.city = city
    def greet(self):
        print(f'Hello! I am {self.name}, {self.age}, from {self.city}.')

s1 = Student('Priya Sharma', 20, 'Mumbai')
s2 = Student('Arjun Mehta',  22, 'Delhi')
s1.greet()   # Hello! I am Priya Sharma, 20, from Mumbai.
s2.greet()   # Hello! I am Arjun Mehta, 22, from Delhi.

# Objects are independent — changing s1 does not affect s2
s1.age = 21
print(s1.age, s2.age)   # 21  22
```

Common Questions

Question	Answer
Why must self be first?	Python passes the object automatically as first arg. Calling s1.greet() is really Student.greet(s1).
Can I name it something other than self?	Technically yes, but never do it — self is universal convention.
When does __init__ run?	Immediately when you write ClassName(...). You cannot prevent it.

STEP 3 — Instance vs Class Variables (0:40 – 0:55)

What to Say

"There are two kinds of variables in a class. Instance variables — defined with self — are unique to each object. Class variables — defined at the top of the class body — are shared across every object. Use class variables for constants or counters that belong to the class concept, not to any one object."

```
class BankAccount:
    bank_name      = 'Dhan Bank of India'  # class variable (shared)
    total_accounts = 0                      # shared counter

    def __init__(self, owner, initial_deposit=0):
        self.owner   = owner            # instance variable (unique)
        self.balance = initial_deposit
        BankAccount.total_accounts += 1

    def deposit(self, amount):
        self.balance += amount
        print(f'Balance: Rs {self.balance}')

acc1 = BankAccount('Priya', 10000)
acc2 = BankAccount('Arjun',  5000)
print(BankAccount.total_accounts)  # 2
acc1.deposit(2500)                 # Balance: Rs 12500
```

Gotcha — Mutable Class Variables

NEVER use a list or dict as a class variable if objects will modify it.
class Bad: members = [] # ALL instances share this same list — bug!
Fix: self.members = [] inside __init__ so each object gets its own list.

STEP 4 — Instance Methods (1:05 – 1:20)

After break

What to Say

"Instance methods are the behaviour of your objects. They can read and modify instance variables, and they can call other methods on the same object using self. Notice in the Rectangle example how describe() calls self.area() and self.is_square() — methods calling methods. This is how you build rich behaviour cleanly."

```
class Rectangle:
    def __init__(self, width, height):
        self.width  = width
        self.height = height

    def area(self):       return self.width * self.height
    def perimeter(self):  return 2 * (self.width + self.height)
    def is_square(self):  return self.width == self.height

    def scale(self, factor):   # modifies in place
        self.width  *= factor
        self.height *= factor

    def describe(self):   # calls other methods via self
        shape = 'Square' if self.is_square() else 'Rectangle'
        print(f'{shape}: {self.width}x{self.height}')
        print(f'  Area = {self.area()}, Perimeter = {self.perimeter()}')

r = Rectangle(6, 4)
r.describe()   # Rectangle: 6x4 | Area = 24, Perimeter = 20
r.scale(2)
r.describe()   # Rectangle: 12x8 | Area = 96, Perimeter = 40
```

STEP 5 — Guided Examples (1:20 – 1:40)

What to Say

"Let's walk through three real-world examples together. I'll write them live — you follow along in your own file. After each one, predict the output before I run it."

Examples in day10_oop_classes.py

Class	What It Models	Key Concepts Used
MovieTicket	Cinema ticket with pricing	Class variable (base_price), _price() helper
CricketPlayer	Batting stats tracker	Accumulator pattern, batting_average()
GroceryCart	Shopping cart with receipt	Dict instance variable, itemised total()

Instructor Tip

Ask students to add a method to each class after you demonstrate.
E.g.: Add a cancel() method to MovieTicket, or a high_scorer() method to CricketPlayer. 5 minutes of unguided extension cements the learning.

STEP 6 — Common Mistakes (1:40 – 1:50)

Mistake	Symptom	Fix
Missing self in method def	TypeError: takes 0 positional arguments	Always start with def method(self, ...)
name = name in __init__	AttributeError on method call	Use self.name = name
Mutable class variable	All objects share same list	Use self.items = [] in __init__
Calling method without ()	Prints <bound method ...>	Use obj.method() with parentheses

STEP 7 — Mini-Challenge: Library Book System (1:50 – 2:00)

What to Say

"For the last 10 minutes, build a Book class entirely on your own. The spec is in the file. I'm walking around — raise your hand if stuck. We'll review the solution together at 2:00."

Challenge Spec at a Glance

Method / Attribute	Behaviour
__init__(title, author, total_copies)	Sets title, author, total_copies, available_copies=total_copies
checkout(member_name)	If copies available: decrease count, print confirmation. Else: print unavailable.
return_book()	If copies out: increase count, print new total. Else: print 'all in library'.
status()	Print: '<title> by <author> — X/Y copies available'

SESSION WRAP-UP

Student Checklist — Before You Leave

- ■ I can write a class with __init__ from scratch
- ■ I understand what self refers to in a method
- ■ I can explain the difference between instance and class variables
- ■ I wrote at least two instance methods that use self
- ■ I completed (or mostly completed) the Library Book mini-challenge
- ■ I know the three common OOP mistakes and how to avoid them

Key Concepts Recap

Concept	One-line summary
class	Blueprint / template for creating objects
object / instance	A concrete thing built from a class
__init__	Constructor — runs automatically on object creation
self	Reference to the current instance; always first param
Instance variable	self.x — unique to each object
Class variable	Defined in class body — shared across all instances
Instance method	Function inside a class that receives self

Day 11 Preview — OOP II

Inheritance: child classes that reuse and extend parent classes
super(): calling the parent's __init__ or methods
Polymorphism: different classes responding to the same method call
Dunder methods: __str__, __repr__, __len__, __eq__ — making objects Pythonic

Troubleshooting Notes

Problem	Likely Cause	Fix
TypeError: greet() takes 0 args	self missing in definition	def greet(self):
AttributeError: 'Dog' has no attribute 'name'	name=name in __init__	self.name = name
All objects show same list	Mutable class variable	Move to self.items = [] in __init__
object at 0x... printed	Missing __str__ / print()	Add print() around return value

QUICK REFERENCE CARD — Day 10

Class Anatomy at a Glance

```
class ClassName:                    # PascalCase
    class_var = 'shared value'      # class variable

    def __init__(self, param):      # constructor
        self.instance_var = param   # instance variable

    def method(self):               # instance method
        return self.instance_var    # access via self

obj = ClassName('hello')           # create object
obj.method()                       # call method
obj.instance_var                   # read attribute
ClassName.class_var                # read class variable
```

Instance vs Class Variables — Decision Table

Question	Use
Is it different for each object?	Instance variable (self.x)
Is it the same constant for all objects?	Class variable
Is it a counter of all objects created?	Class variable (increment in __init__)
Is it a mutable collection (list/dict)?	ALWAYS instance variable (self.x = [])

VS Code Shortcuts

Action	Shortcut
Run file	Ctrl+F5 (Windows/Linux) | Cmd+F5 (Mac)
Open terminal	Ctrl+` (backtick)
Go to definition	F12
Rename symbol	F2
Format document	Shift+Alt+F